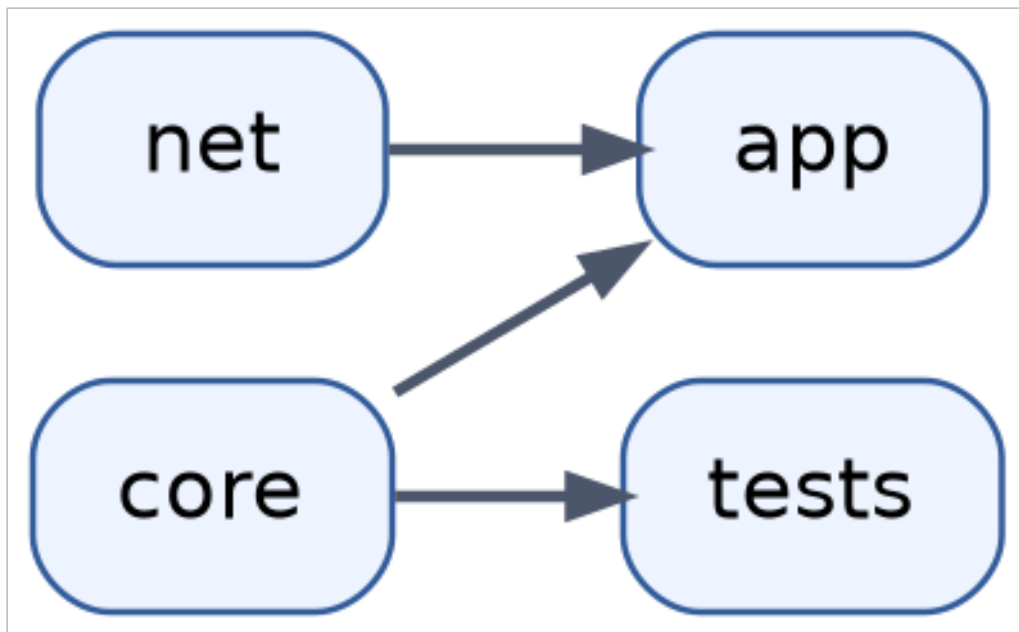


Build Order with Cycle Detection

Kahn's Topological Sort in C++20

Interview study guide based on Task 006



The dependency graph is stored in prerequisite -> dependent direction.

1. What the problem is asking

Given a list of build targets and dependency pairs {target, prerequisite}, return every target exactly once in an order where each prerequisite appears before the target that depends on it. Invalid input, duplicate target names, unknown names, and dependency cycles must return `std::nullopt`.

The deterministic rule is important: whenever several targets are buildable at the same time, choose the lexicographically smallest name.

Core mental model

- Treat each target as a vertex in a directed graph.
- For a dependency {target, prerequisite}, store the graph edge prerequisite -> target.
- The in-degree of a target is the number of prerequisites that are still unresolved.
- A target with in-degree 0 can be built immediately.
- After building a target, decrease the in-degree of each target that depends on it.
- If not every target can be processed, the remaining graph contains a cycle.

Why the edge direction matters

```
Input pair: {"app", "core"}  
Meaning:   core must be built before app  
Stored edge: core ---> app
```

This direction lets the algorithm answer the useful question: after I build core, which targets should have one prerequisite removed?

2. The two main data structures

Adjacency list

```
std::unordered_map<std::string,  
                  std::unordered_set<std::string>> adjacency;
```

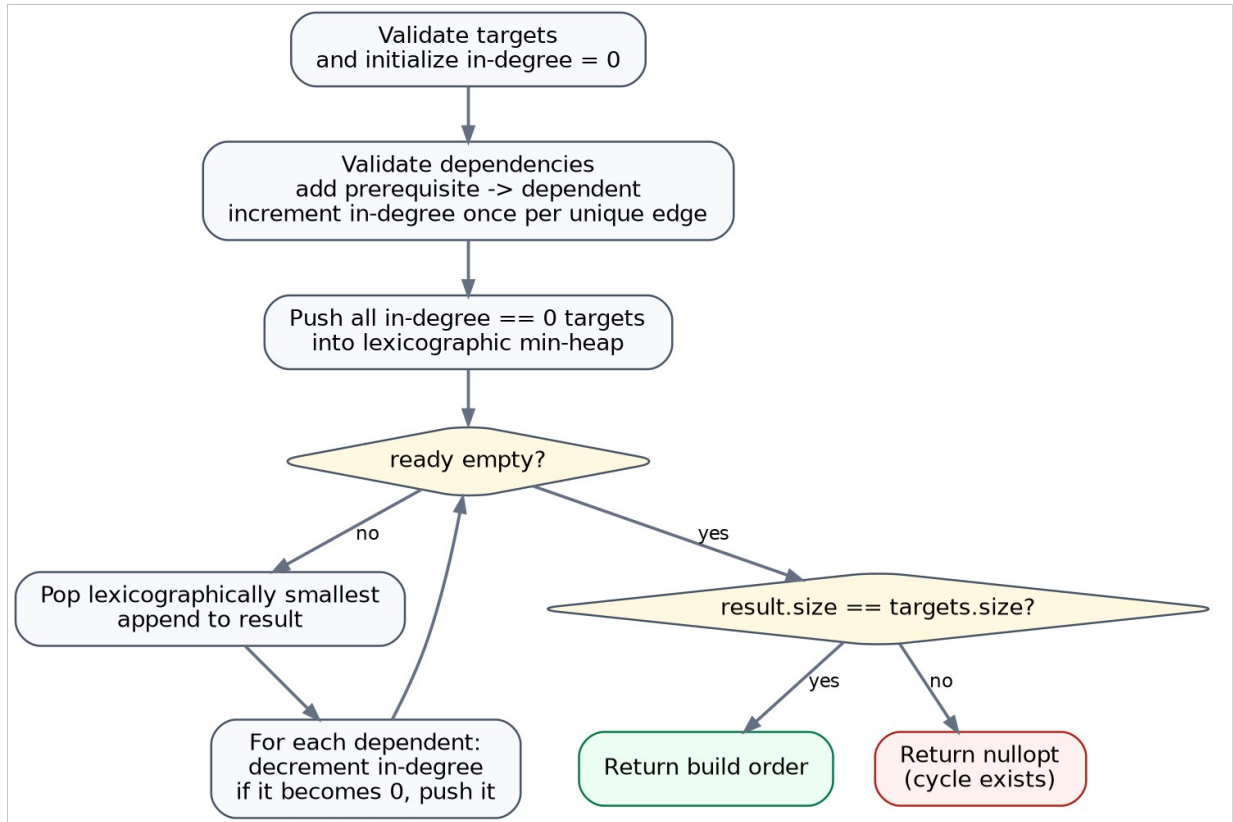
`adjacency[prerequisite]` contains the targets that directly depend on that prerequisite. The `unordered_set` is intentional: duplicate dependency edges must be ignored rather than counted twice.

In-degree table

```
std::unordered_map<std::string, std::size_t> in_degree;
```

`in_degree[x]` is the number of unresolved prerequisites for x. Initially every target has degree 0. Each unique dependency increments the target degree once.

3. Full algorithm



Kahn's algorithm with validation and a lexicographic min-heap.

Invariant to remember

At every iteration, every target in ready has in-degree 0, meaning all of its prerequisites have already been processed. For every unprocessed target, in_degree stores exactly how many prerequisites remain unresolved.

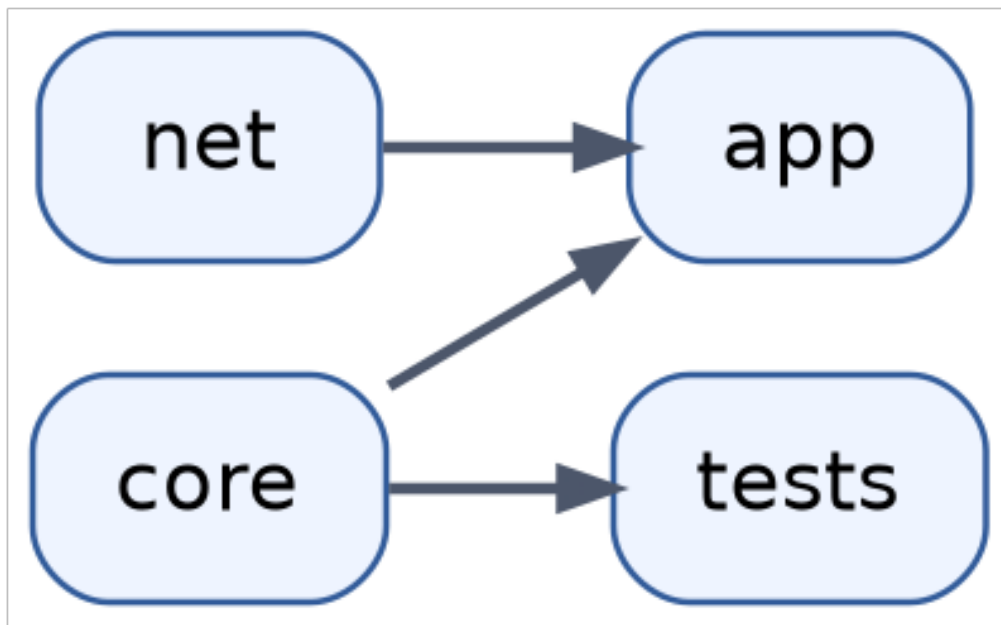
Why a min-heap?

```
std::priority_queue<
    std::string,
    std::vector<std::string>,
    std::greater<>
> ready;
```

`std::priority_queue` is normally a max-heap. `std::greater<>` turns it into a min-heap, so if app, core, and tests are all ready, app is selected first. This makes the result deterministic.

4. Step-by-step walkthrough

Example targets: app, core, net, tests. Dependencies: app needs core, app needs net, tests needs core.



Stored directed edges: core -> app, net -> app, core -> tests.

Initial state

Target	Initial in-degree	Reason	Initially ready?
app	2	needs core and net	No
core	0	no prerequisite	Yes
net	0	no prerequisite	Yes
tests	1	needs core	No

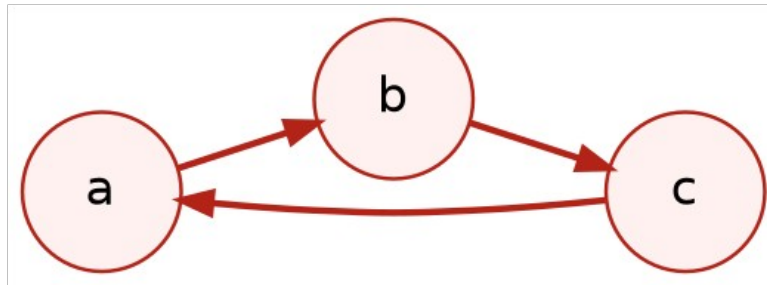
The heap starts with core and net. Because core is lexicographically smaller, core is selected first.

State transitions

Step	Build	Changed degrees	Ready after update	Result
1	core	app 2->1; tests 1->0	net, tests	core
2	net	app 1->0	app, tests	core, net
3	app	none	tests	core, net, app
4	tests	none	empty	core, net, app, tests

Notice the subtle point at step 2: tests became ready earlier, but app becomes ready after net is processed. The heap then contains app and tests, and app wins lexicographically.

5. Cycle detection



Example cycle: a -> b -> c -> a.

In a pure cycle, every vertex has in-degree at least 1. Therefore no vertex enters the ready heap. More generally, an acyclic part may be processed first, but vertices trapped in a cycle never reach in-degree 0.

```
if (result.size() != targets.size()) {  
    return std::nullopt;  
}
```

This size check is the cycle detector. If all targets were processed, there is no cycle. If some remain, they cannot become buildable because they participate in, or are blocked by, a cycle.

Self-cycle

```
targets = {"a"}  
dependencies = {"a", "a"}  
  
in_degree["a"] == 1  
ready is empty  
=> result remains empty  
=> return std::nullopt
```

6. Input validation and duplicate handling

Duplicate target names

```
const auto [it, inserted] = in_degree.emplace(target, 0);  
if (!inserted) {  
    return std::nullopt;  
}
```

`unordered_map::emplace` reports whether a new key was inserted. A second occurrence of the same target therefore makes the input invalid immediately.

Unknown names

```
if (!in_degree.contains(target) ||  
    !in_degree.contains(prerequisite)) {  
    return std::nullopt;  
}
```

```
}
```

Both names in every dependency must already exist in targets.

Duplicate dependency edges

```
auto& dependents = adjacency[prerequisite];
const auto [it, inserted] = dependents.insert(target);

if (inserted) {
    ++in_degree[target];
}
```

The `unordered_set` suppresses duplicate edges. This is essential: if `{b, a}` appeared twice and the degree were incremented twice, `b` would incorrectly remain blocked after `a` is built.

7. Complexity

Let V be the number of targets and E the number of unique dependency edges.

Work	Cost	Why
Build maps/sets	Average $O(V + E)$	Hash-table insertions are average constant time.
Heap operations	$O(V \log V)$	Each target enters and leaves ready at most once.
Traverse edges	$O(E)$	Each unique edge is considered once.
Total	$O(V + E + V \log V)$	Within the requested $O((V + E) \log V)$.
Extra space	$O(V + E)$	Graph, degree table, heap, result.

Interview nuance: the `unordered_map/unordered_set` statements are average-case complexity. The stated task target is still satisfied by this design under normal hash-table assumptions.

8. Interview explanation in 30 seconds

I model each dependency as `prerequisite -> target` and count each target's unique prerequisites as its in-degree. I push all zero-in-degree targets into a min-heap, so the lexicographically smallest buildable target is always chosen. Each time I remove one, I decrement the in-degree of its dependents and push any that reach zero. Duplicate edges are stored in an `unordered_set` so they do not inflate in-degree. If I process fewer vertices than the number of targets, some vertices never reached zero in-degree, which means the dependency graph contains a cycle.

9. Complete C++20 code

The implementation below follows the submitted solution. Only presentation-level cleanup is applied: the duplicate `<cassert>` include and an unfinished commented debug line are removed, and unused structured-binding iterators are avoided. The algorithm and test coverage are unchanged.

```
#include <cassert>
#include <functional>
#include <iostream>
#include <optional>
```

```

#include <queue>
#include <string>
#include <unordered_map>
#include <unordered_set>
#include <utility>
#include <vector>
// Solution: Kahn's topological-sort algorithm with a min-heap

std::optional<std::vector<std::string>> buildOrder(
    const std::vector<std::string>& targets,
    const std::vector<std::pair<std::string, std::string>>& dependencies)
{
    // adjacency[prerequisite] contains targets that depend on it.
    std::unordered_map<std::string, std::unordered_set<std::string>> adjacency;

    // Number of prerequisites that each target still has.
    std::unordered_map<std::string, std::size_t> in_degree;

    // First build the set of valid targets and detect duplicate names.
    for (const auto& target : targets) {
        const bool inserted = in_degree.emplace(target, 0).second;
        if (!inserted) {
            return std::nullopt;
        }

        adjacency.emplace(target, std::unordered_set<std::string>{});
    }

    // A pair {target, prerequisite} means:
    //
    // prerequisite ---> target
    //
    // Duplicate edges are ignored by the unordered_set.
    for (const auto& [target, prerequisite] : dependencies) {
        if (!in_degree.contains(target) ||
            !in_degree.contains(prerequisite)) {
            return std::nullopt;
        }

        auto& dependents = adjacency[prerequisite];

        const bool inserted = dependents.insert(target).second;

        if (inserted) {
            ++in_degree[target];
        }
    }

    // Min-heap: among all currently buildable targets,
    // always select the lexicographically smallest one.
    std::priority_queue<
        std::string,
        std::vector<std::string>,
        std::greater<>
    > ready;

    for (const auto& target : targets) {
        if (in_degree[target] == 0) {
            ready.push(target);
        }
    }

    std::vector<std::string> result;
    result.reserve(targets.size());

    while (!ready.empty()) {
        std::string current = ready.top();
        ready.pop();

        result.push_back(current);

        for (const auto& dependent : adjacency[current]) {
            auto& degree = in_degree[dependent];

```

```

        --degree;

        if (degree == 0) {
            ready.push(dependent);
        }
    }
}

// If some targets could never become buildable, they belong
// to a dependency cycle.
if (result.size() != targets.size()) {
    return std::nullopt;
}

return result;
}

int main()
{
    {
        const std::vector<std::string> targets;
        const std::vector<std::pair<std::string, std::string>> dependencies;

        const auto result = buildOrder(targets, dependencies);

        assert(result.has_value());
        assert(result->empty());
    }

    {
        const std::vector<std::string> targets{"app"};
        const std::vector<std::pair<std::string, std::string>> dependencies;

        const auto result = buildOrder(targets, dependencies);

        assert(result == std::optional<std::vector<std::string>>{
            {"app"}
        });
    }

    {
        const std::vector<std::string> targets{"c", "a", "b"};
        const std::vector<std::pair<std::string, std::string>> dependencies;

        const auto result = buildOrder(targets, dependencies);

        assert((result == std::optional<std::vector<std::string>>{
            {"a", "b", "c"}
        })));
    }

    {
        const std::vector<std::string> targets{
            "app", "core", "net", "tests"
        };

        const std::vector<std::pair<std::string, std::string>> dependencies{
            {"app", "core"},
            {"app", "net"},
            {"tests", "core"}
        };

        const auto result = buildOrder(targets, dependencies);

        assert((result == std::optional<std::vector<std::string>>{{"core", "net", "app", "tests"}}));
    }

    {
        const std::vector<std::string> targets{
            "a", "b", "c", "d"
        };
    }
}

```



```

const std::vector<std::pair<std::string, std::string>> dependencies{
    {"b", "a"},
    {"c", "b"},
    {"d", "c"}
};

const auto result = buildOrder(targets, dependencies);

assert((result == std::optional<std::vector<std::string>>{"a", "b", "c", "d"}));
}

{
    // Duplicate targets are invalid.
    const std::vector<std::string> targets{"a", "b", "a"};

    const auto result = buildOrder(targets, {});

    assert(!result.has_value());
}

{
    // Duplicate dependency must count only once.
    const std::vector<std::string> targets{"a", "b"};

    const std::vector<std::pair<std::string, std::string>> dependencies{
        {"b", "a"},
        {"b", "a"}
    };

    const auto result = buildOrder(targets, dependencies);

    assert((result == std::optional<std::vector<std::string>>{"a", "b"}));
}

{
    // Unknown target.
    const std::vector<std::string> targets{"a", "b"};

    const std::vector<std::pair<std::string, std::string>> dependencies{
        {"c", "a"}
    };

    assert(!buildOrder(targets, dependencies).has_value());
}

{
    // Unknown prerequisite.
    const std::vector<std::string> targets{"a", "b"};

    const std::vector<std::pair<std::string, std::string>> dependencies{
        {"b", "x"}
    };

    assert(!buildOrder(targets, dependencies).has_value());
}

{
    // Self-cycle.
    const std::vector<std::string> targets{"a"};

    const std::vector<std::pair<std::string, std::string>> dependencies{
        {"a", "a"}
    };

    assert(!buildOrder(targets, dependencies).has_value());
}

{
    // Longer cycle:
    //
    // a -> b -> c -> a
    const std::vector<std::string> targets{"a", "b", "c"};

```

```

    const std::vector<std::pair<std::string, std::string>> dependencies{
        {"b", "a"},
        {"c", "b"},
        {"a", "c"}
    };

    assert(!buildOrder(targets, dependencies).has_value());
}

std::cout << "All tests passed\n";
}

```

10. What to remember

- Dependency pair {target, prerequisite} becomes graph edge prerequisite -> target.
- In-degree means unresolved prerequisite count.
- Zero in-degree means buildable now.
- A min-heap provides deterministic lexicographic choice.
- An unordered_set prevents duplicate edges from inflating in-degree.
- After processing one target, decrement only its direct dependents.
- If processed target count is smaller than V, a cycle prevented completion.

The hardest part is not the C++ syntax. It is keeping the graph direction and the meaning of in-degree clear. Once those are correct, the rest of Kahn's algorithm follows naturally.